

Layered Fluid Simulation with MPM

Recreating stylized particle fluids with continuum physics
Inspired By Grant Kot's Liquid Layers

This project is inspired by Grant Kot's *Liquid Layers*, which is a visually rich particle-based fluid simulation. My goal was to recreate similar layered fluid behavior—but using the Material Point Method, which is a continuum-based simulation framework.

Motivation

Can physics-based simulation reproduce stylized behavior?

Particle simulators are:

- visually expressive
- not physically grounded

Goal: achieve

- layering
- mixing
- Interactivity

using MPM (continuum physics)

Abstract

We present a new particle-based method for viscoelastic fluid simulation. We achieve realistic small-scale behavior of substances such as paint or mud as they splash on moving objects. Incompressibility and particle anti-chattering are enforced with a double density relaxation procedure which updates particle positions according to two opposing pressure terms. From this process surface tension effects emerge, enabling drop and filament formation. Elastic and non-linear plastic effects are obtained by adding springs with varying rest length between particles. We also extend the technique to handle interaction between fluid and dynamic objects. Various simulation scenarios are presented including rain drops, fountains, clay manipulation, and floating objects. The method is robust and stable, and can estimate splashing behavior at interactive frame-rates.

Categories and Subject Descriptors (according to ACM CCS): I.3.5 [Computer Graphics]: Computational Geometry and Object Modeling – Physically Based Modeling; I.3.7 [Computer Graphics]: Three Dimensional Graphics and Realism – Animation and Virtual Reality

1. Introduction

The field of computational fluid dynamics has introduced many different algorithms to simulate the complex behavior of fluids. Since liquids and gases play an important role in everyday life, several fluid simulation methods have been successfully used for computer graphics applications.

Two main categories of simulation methods exist: Eulerian grids and Lagrangian particles. Eulerian methods discretize the problem using a subdivision of the spatial domain and control fluid flow in each cell. Such techniques have been able to achieve among the most realistic simulations of liquid types ranging from simple flows to highly viscous fluids, with plastic and elastic behaviors. They easily enforce the incompressibility condition, but do not guarantee mass conservation for small features.

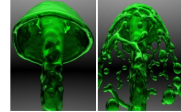
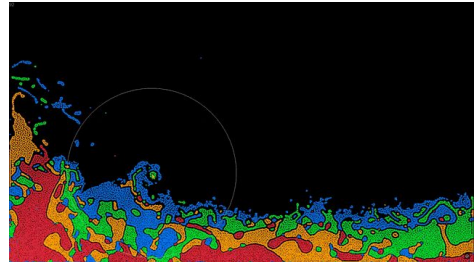


Figure 1: Frontview simulation illustrating sheet, filament, and drop formation.



The core question is whether a physically-based simulation can reproduce these stylized behaviors—layering, mixing, and interaction—without directly scripting particle forces.

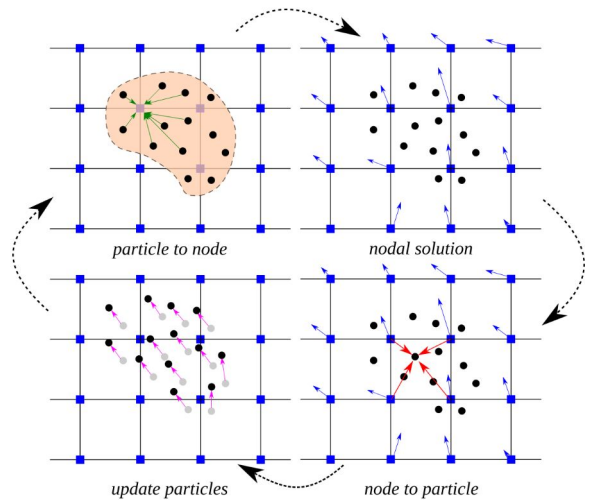
Particle simulations like Kot's are visually appealing, but behavior comes from forces applied directly between particles which doesn't necessarily follow physical laws. I wanted to see if a physically-based method like MPM could reproduce similar effects like layering and mixing—while remaining grounded in continuum mechanics, where particles represent samples of a continuous material, and interactions are computed on a grid.

TLDR; Same visual goal, completely different physics model: recreate this kind of layered fluid behavior using MPM instead of particle forces

Material Point Method (MPM) Intuition

Particles don't directly interact — the grid mediates interactions

- particles carry material state
- grid computes physics forces
- transfer back and move

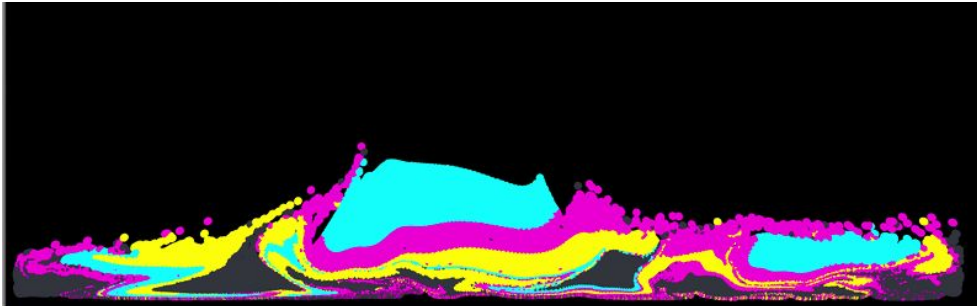


Credit: [Berkeley Geomechanics Group](#)

MPM is a hybrid method. Particles carry mass and material state, but all interactions happen on a background grid. Each step transfers data to the grid, computes forces, then transfers back. So particles don't directly interact—their interaction is mediated through the grid. This creates smoother, more physically consistent behavior compared to direct particle forces.

Implementation & Modeling Decisions

- Constitutive model: weakly compressible fluid (Tait EOS)
- Simplifications:
 - no strict incompressibility
 - grid-based pressure
 - parameter-based layering
- Two modes: interactive demo + random stress test

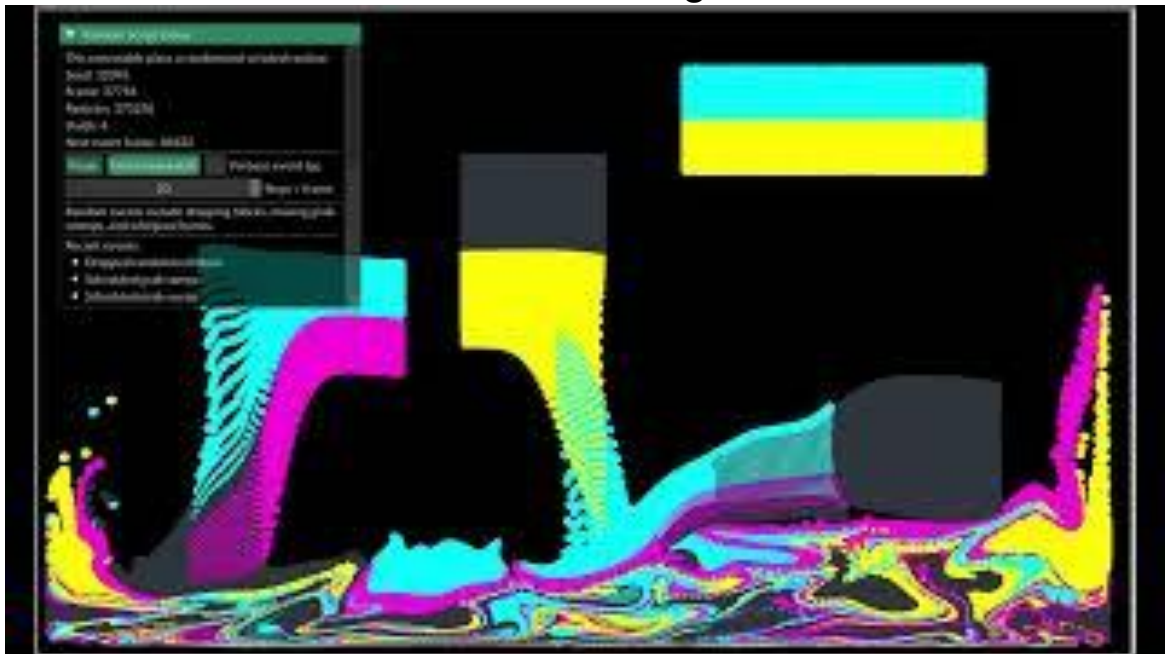


To keep the simulation efficient and simple, I made several design choices in my implementation: I use a weakly compressible model, compute pressure on a grid, and control layering through material parameters. Each of which introduces tradeoffs in physical realism.

- Instead of enforcing exact incompressibility the weakly compressible model penalizes density changes rather than preventing them entirely.”
- All pressure forces are computed on a background grid, which smooths interactions and mediates motion between particles due to grid-particle interpolation and grid resolution
- Layering in my simulation is controlled through material parameters like density, rather than emerging purely from a fully resolved physical model

I also built two modes: an interactive version for exploration and a randomized version for stress testing. The interactive version deliberately allows unstable parameter choices—so users can see both correct behavior and failure modes.”

Demo 1: Random Forcing Stress Test

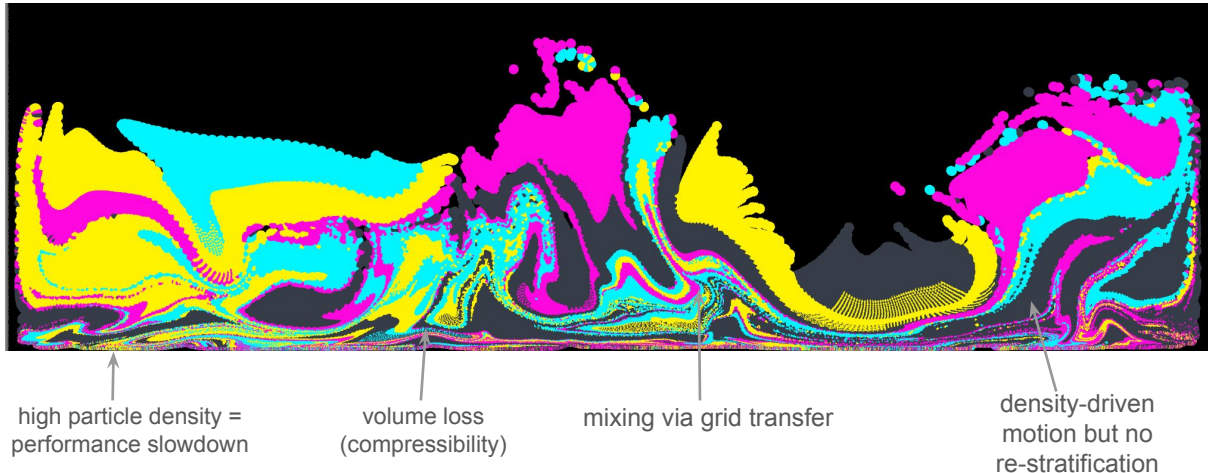


This is a randomized stress test where I repeatedly inject forces and particles through block drops and whirlpool or grab interactions to stress the solver. The simulation remains stable despite increasingly aggressive perturbations under the randomized parameter regime. There are no explosions, fluid motion is smooth, and realistic mixing occurs. The eventual bottleneck is performance as particle count grows, not numerical blow up

stability depends strongly on:

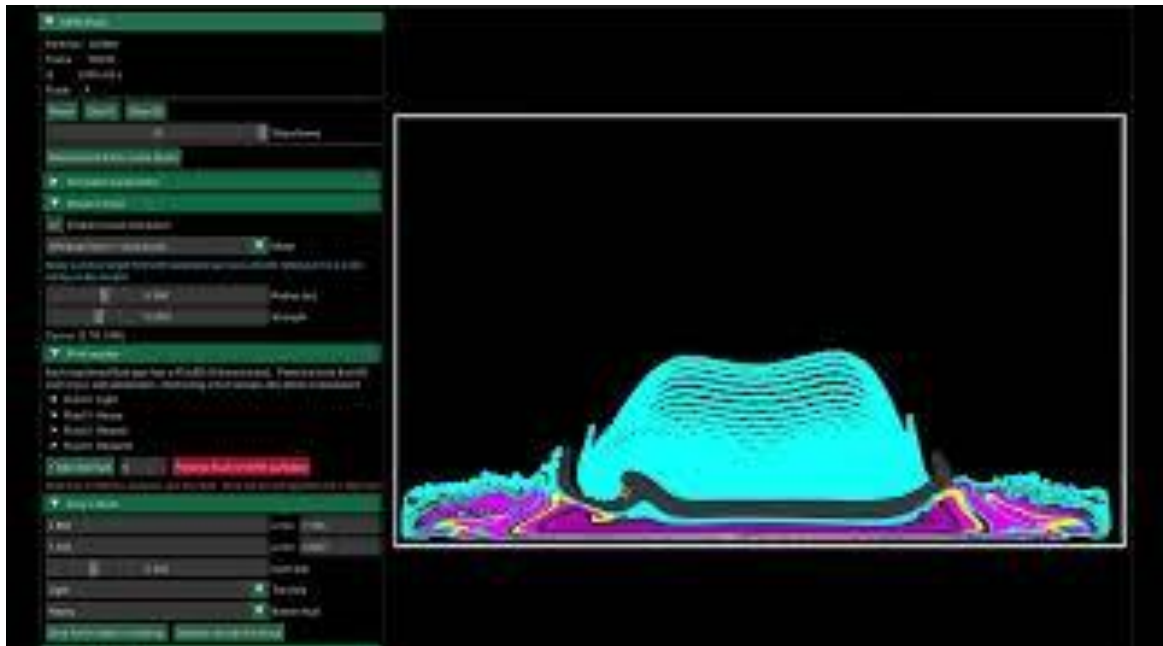
- stiffness / EOS parameter k
- steps per frame
- force strength
- particle count
- boundary handling

Artifact Analysis



This frame shows both the strengths and limitations of the simulation. In the center, we see mixing that arises from grid-based interaction—particles don't directly interact, but exchange momentum through the grid. You can see volume loss due to the weakly compressible model particularly near the boundaries and areas where dropped blocks have made contact. This also leads to high particle densities. On the right, you can see density-influenced motion and layering, where heavier fluid settles below lighter fluid but there is no re-stratification after mixing due to numerical diffusion from grid-particle interpolation (grid resolution smooths out details) and the weakly compressible model. These artifacts redirect consequences of the model, so this frame highlights the both the plausible physical behavior and numerical limitations of my implementation.

Demo 2: Interactive Behavior

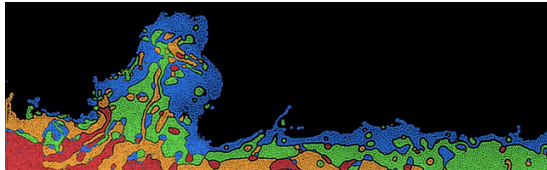


Here's the interactive simulation. I start with a single fluid, then proceed by adding liquids of different densities and applying mouse forces. When I apply a localized force, the motion is mediated through the grid, producing smooth mixing. While you can see that density differences influence fluid motion, the simulation does not produce sharp re-stratification after mixing due to numerical diffusion and the weakly compressible model. If I keep adding energy and particles, the simulation eventually slows down and starts to show artifacts, which highlights both performance limits and the sensitivity to parameters. Towards the end, I experimented with aggressive forcing to see what would happen once parameters left the stable regime. My implementation is not unconditionally stable. It is stable for useful parameter ranges, but one point of the interactive demo is that users can push parameters outside that range and see how the numerical method fails.

Takeaway

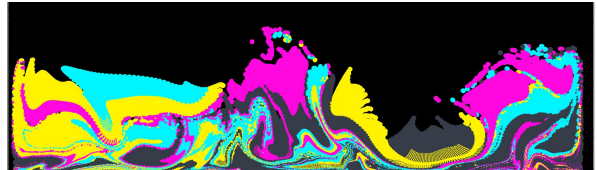
Visual realism $\neq$ physical realism

Kot's work (Clavet et. al)



- continuum-inspired but constraint-driven
- sharp, stylized motion
- artist-driven

My work (MPM)



- continuum-based and equation-driven
- smoother behavior
- physically interpretable
- computationally expensive

The key takeaway is that there are 2 axes: Physical realism (how faithfully you solve the true equations) and **Visual realism** (how convincing it *looks*) and importantly, **Improving one does not guarantee improving the other. A simulation can look realistic without being physically accurate and can be physically accurate without looking good.**

The outcome of my implementation was that I **achieved *plausible fluid behavior*, but not all expected physical phenomena (like clean re-stratification).**

So while Kot's simulation is particle-force based and visually crisp, Mine uses MPM and is continuum-based, which makes it smoother and more physically grounded, but also more computationally expensive.

Overall, Matching stylized visuals with physics is not trivial, and that's where a lot of interesting challenges arise. Effects like buoyancy and surface tension are physically grounded, but in many MPM simulations they are approximated or added explicitly to enhance visual behavior. In my implementation, most behavior comes from pressure and density differences, but future work could be adding additional forces could be added to make effects like separation or cohesion more pronounced.